

Observation Kalman Filtering

Denoising observed smiles without confusing noise with gaps

Vol-Fitter Technical Notes, No. 15

Vol-Fitter

July 4, 2026

Abstract

Prior persistence and Kalman filtering both pull a smile away from a raw, current-market fit, so they are easy to confuse in implementation. They must not be confused mathematically. Prior persistence (Note 13) is a *gap regularizer*: it keeps unobserved or weakly identified smile functionals near a transported prior and turns itself off where today's quotes already identify the functional. The observation filter of this note is a *temporal state estimator*: it assumes the current smile is observed through noisy, sometimes contradictory quotes, predicts the smile from the previous filtered state under the SSR transport of Note 12, and updates according to the relative precision of prediction and observation. The state is not raw option mids but the compact arbitrage-safe ATM handles of Note 01; the measurement covariance is propagated from the data-only fit's solution Jacobian through the handle map, inflated by realized inconsistency; and active mode is a one-stage MAP calibration so the same quotes are never counted twice. The machinery is in production, and the note now reports what it measured. In the canonical case file a stale-strike curvature kink is accepted at gain 0.027 while the genuine level move passes at gain 0.80. A 38 181-step temporal backtest over three market regimes flipped the clock process noise from the design value of 10 to 30 vol bp per $\sqrt{\text{day}}$ (standardized residuals collapse toward 1 in every regime, shock lag shrinks 3–7 $\times$), confirmed the Jacobian-propagated covariance as the default route (2–3 $\times$ better contradiction rejection than the factor route in every regime), and taught the one genuinely structural lesson: on coarse-strike chains the full-covariance update let a junk curvature innovation drag the ATM level through off-diagonal gains, so production runs the update per handle.

Contents

1	The production symptom	2
2	Two regularizers, two measures	2
2.1	Prior persistence	3
2.2	Observation filtering	3
3	The Kalman object	3
4	Where the observation covariance comes from	5
4.1	Do not filter raw mids	5
4.2	Extract a noisy handle observation	5
4.3	The Jacobian route (default)	5

4.4	Units: information is only information in stated noise	6
4.5	The factor route (fallback and A/B column)	6
5	The prediction law and the state's life cycle	7
5.1	Transport and process noise	7
5.2	Seeding, resets, and what survives a recalibration	8
6	The clean architecture	8
6.1	Overlay mode: compute, display, do not affect calibration	8
6.2	Active mode: one-stage MAP, not posterior plus quotes	9
6.3	Where prior persistence remains	10
7	Two case files	10
8	What the backtest measured	12
9	What is original here	14
10	Limitations	14
11	Traceability	15
A	Hyperparameter atlas	15
B	Performance notes	16
C	Reference implementation	17

1 The production symptom

A sparse wing and a noisy cluster are different failures.

In a sparse wing, the desk has almost no market information. A data-only fit can extend the ATM skew into the tail and create a new signal where no option actually traded. Note 13 solved that problem with an activation gate: the prior is strong in the gap and exactly off where the data is sufficient.

In a noisy cluster, the desk has information, but it is internally inconsistent. Two adjacent strikes may imply a local kink because one market is stale, a wide bid-ask band may make the mid meaningless, or an American wing repair (Note 05) may have preserved spread while lowering the reliable rank of the wing. The right answer is not “fill with yesterday”; the right answer is “infer the latent current smile under observation noise.”

Invariant

1. Prior persistence acts only on functionals not identified by the current market at the required precision.
2. Observation filtering acts only through an explicit prediction covariance and an explicit measurement covariance.
3. The same quote cannot be used once to create a filtered posterior and then again as an independent calibration datum (proposition 2).
4. The filter state is an arbitrage-safe coordinate — the exact ATM handles of Note 01 — never a free raw-IV curve and never native local-vol parameters.
5. Every filtered output reports prediction, observation, innovation, gain and posterior uncertainty.

The two features therefore answer two different questions:

Prior persistence: Where do we lack enough observations to identify the smile?

Kalman filtering: Given observations here, how noisy are they?

The implementation preserves that distinction all the way to the UI, and the temporal backtest of section 8 scores it: the success criterion is lower held-out error on *noisy* snapshots with no meaningful lag on high-quality moves — explicitly not “lower every RMS”.

2 Two regularizers, two measures

Let $x_t \in \mathbb{R}^d$ denote the latent smile state of one node at snapshot time t : in production this is the compact handle vector

$$x_t = (\sigma_{\text{atm},t}, s_{\text{atm},t}, \kappa_{\text{atm},t})^T,$$

the same three coordinates the graph extrapolator of Note 14 propagates (signed operators such as RR25, BF25 and var-swap are a dimension-agnostic extension left for after the pilot). Let q_t be the prepared quote set after forwards, de-Americanization, event-clock conversion, bid-ask band construction and quote edits.

2.1 Prior persistence

Prior persistence is a penalty in a calibration objective. With a prior state x_t^0 transported to the current forward, and functionals $O_j : \mathbb{R}^d \rightarrow \mathbb{R}$, Note 13 has the universal form

$$\mathcal{L}_{\text{persist}}(x) = \frac{1}{2} \sum_j \lambda_j^{\text{gap}} \left(\frac{O_j(x) - O_j(x_t^0)}{a_j} \right)^2, \quad \lambda_j^{\text{gap}} = \lambda_j^0 \max \left(1 - \frac{\pi_j^{\text{obs}}}{\pi_j^{\text{req}}}, 0 \right)^\gamma. \quad (1)$$

The gate is driven by *identifiability*: when current quotes identify functional O_j at or above the required precision, $\lambda_j^{\text{gap}} = 0$. Thus prior persistence is not primarily a denoiser. It is a rule for refusing to invent signal outside the support of the observation.

2.2 Observation filtering

A Kalman filter is instead a sequential estimator. It starts from the previous posterior law for x_{t-1} , transports it forward, then combines the transported prediction with today's noisy observation. In the linear-Gaussian handle model,

$$x_t = \Phi_t x_{t-1} + u_t + \eta_t, \quad \eta_t \sim \mathcal{N}(0, Q_t), \quad (2)$$

$$z_t = H_t x_t + \epsilon_t, \quad \epsilon_t \sim \mathcal{N}(0, R_t), \quad (3)$$

where z_t is not a raw quote vector; it is a noisy handle estimate extracted from today's prepared quotes. Q_t is process covariance: time elapsed, spot move, event risk and transport distance. R_t is measurement covariance: bid–ask width, quote density, stale fields, de-Americanization repairs, band slack and contradictory close strikes.

Heuristic

Persistence says “do not put signal where the market did not speak.” Filtering says “when the market speaks with a noisy voice, infer what it probably meant.” Both can pull toward a previous surface, but the sufficient statistic is different: persistence uses a *gap*; filtering uses a *covariance*.

3 The Kalman object

The Vol-Fitter implements the filter in covariance form, matching the graph posterior style of Note 14. For a direct handle observation $H_t = I$ the form is especially transparent, but the equations allow a subset or a linear operator measurement.

Definition 1 (Predicted and observed handle laws). At snapshot t , the prediction law and the measurement law are

$$x_t | \mathcal{F}_{t-1} \sim \mathcal{N}(m_t^-, P_t^-), \quad z_t | x_t \sim \mathcal{N}(H_t x_t, R_t).$$

m_t^- is the transported previous filtered state; P_t^- is its transported uncertainty plus process noise. z_t and R_t are extracted from today's prepared quotes without using prior persistence.

The central update is

$$\begin{aligned}
S_t &= H_t P_t^- H_t^\top + R_t, \\
K_t &= P_t^- H_t^\top S_t^{-1}, \\
m_t^+ &= m_t^- + K_t(z_t - H_t m_t^-), \\
P_t^+ &= (I - K_t H_t) P_t^- (I - K_t H_t)^\top + K_t R_t K_t^\top.
\end{aligned} \tag{4}$$

The last line is the Joseph form; it is a little longer than $P_t^+ = (I - K_t H_t) P_t^-$, but it preserves symmetry and positive semidefiniteness for *any* gain — which is precisely what makes the pilot gain cap below safe: a capped K is still a valid, merely suboptimal, linear gain. The production body is small enough to quote whole:

Listing 1: The production update (quoted from `volfit/calib/observation_filter.py`): eq. (4) with the own-gain pilot cap, Joseph covariance, and an explicit PSD guard.

```

1  innovation = z - H @ m
2  S = H @ P @ H.T + R
3  K = np.linalg.solve(S.T, (P @ H.T).T) # gain without an explicit
    inverse
4  if max_gain < 1.0: # pilot cap: scales OWN-
    gains only
5      own = np.abs(np.diag(K @ H))
6      scale = np.minimum(1.0, max_gain / np.maximum(own, 1e-300))
7      K = K * scale[:, None]
8  out_mean = m + K @ innovation
9  i_kh = np.eye(m.size) - K @ H
10 out_cov = i_kh @ P @ i_kh.T + K @ R @ K.T # Joseph form: PSD for ANY
    gain
11 out_cov = 0.5 * (out_cov + out_cov.T)
12 if np.any(np.linalg.eigvalsh(out_cov) < -1e-12):
13     raise FloatingPointError("posterior covariance lost PSD")

```

Proposition 1 (Precision-weighted shrinkage). *In one dimension with $H_t = 1$, prediction variance p and measurement variance r , the posterior mean is*

$$m^+ = \frac{r}{p+r} m^- + \frac{p}{p+r} z,$$

and the Kalman gain is $K = p/(p+r)$.

Proof. The innovation variance is $S = p + r$, so $K = p/S$. Substituting in $m^+ = m^- + K(z - m^-)$ gives $m^+ = (1 - K)m^- + Kz = r(p+r)^{-1}m^- + p(p+r)^{-1}z$. $\square$

Proposition 1 is the whole desk intuition. If today's observation is tight ($r \ll p$), the filtered smile moves to the market. If today's observation is noisy ($r \gg p$), it stays near the transported prediction. The movement is not controlled by whether the region is observed; it is controlled by the relative uncertainties of prediction and observation.

Remark 1 (Production runs eq. (4) per handle). The design allowed a full $d \times d$ update. Production diagonalizes P_t^- and R_t before the update (`DIAGONAL_UPDATE` in `volfit/api/observation_filter.py`),

so each handle receives exactly the scalar gain of proposition 1 — the same convention as the Note 14 graph posterior. That is a measured decision, not an aesthetic one: the backtest showed the off-diagonal gains of the full update turning a junk curvature innovation into a multi-vol-point ATM error on coarse-strike chains. The second case file of section 7 tells that incident in full; the full-covariance update remains available for later study.

4 Where the observation covariance comes from

The hard part is not the Kalman algebra. The hard part is building a measurement covariance R_t that reflects the quote set honestly. Production has two routes, selected by `filterCovarianceMode`: the default *Jacobian route* propagates the fit’s observed information through the handle map, and the *factor route* reuses the graph layer’s precision vocabulary as a fallback and A/B diagnostic. Both live in `volfit/calib/observation_measurement.py`.

4.1 Do not filter raw mids

Raw contracts are a poor state space. The OTM side changes when the forward moves; contracts appear and disappear; American contracts need early-exercise removal; a strike can be inside a wide spread but far from its midpoint; and a raw implied-vol vector is not guaranteed to remain butterfly-free after smoothing. The filter begins after the existing quote-prep pipeline in `volfit/api/quotes.py`.

4.2 Extract a noisy handle observation

The observation extractor is a two-step map:

$$z_t = \mathcal{H}\left(\arg\min_{\theta} \mathcal{L}_{\text{quote}}(\theta; q_t) + \mathcal{L}_{\text{intrinsic}}(\theta)\right), \quad (5)$$

where $\mathcal{L}_{\text{quote}}$ is the vega-normalized, bid–ask-aware loss of Note 07, $\mathcal{L}_{\text{intrinsic}}$ contains only the intrinsic model regularization needed for a valid no-arbitrage smile (never temporal prior persistence), and $\mathcal{H}$ reads the exact ATM handles off the fitted slice. In production the carrier is the LQD backbone — always fitted, model-agnostic, with closed-form handles (Note 01) — so $z_t = (\hat{\sigma}_{\text{atm}}, \hat{s}_{\text{atm}}, \hat{\kappa}_{\text{atm}})$ costs nothing extra. When the committed fit did receive persistence targets (hybrid mode with an active prior), the measurement is flagged `contaminated` rather than silently trusted; Note 13’s activation gate keeps the overlap near zero exactly where the filter has signal, and the `opt-in filterDataOnlyPrepass` buys a strictly clean z_t at the cost of one extra data-only fit per node.

4.3 The Jacobian route (default)

Near the optimum, linearize the residual stack as $r(\theta) \approx r(\hat{\theta}) + J(\theta - \hat{\theta})$. If the handle map is $x = g(\theta)$ with Jacobian $G = \partial g / \partial \theta$, the observed-information approximation is

$$\mathcal{I}_{\theta} = J^{\top} W J + \Lambda_{\text{intrinsic}}, \quad R_x \approx G \mathcal{I}_{\theta}^{\dagger} G^{\top}. \quad (6)$$

Implementation makes this nearly free. Every parametric calibrator solves via least squares and retains the solution Jacobian in a diagnostics side-channel; the square-root weights and inverse-vega scaling are already folded into its rows, so $J^{\top} J$ is the information in vol units, with the intrinsic

regularization rows (ridge, calendar, barrier) supplying $\Lambda_{\text{intrinsic}}$ automatically. G is a central finite difference of the handle map — slice *builds*, not fits, on the coarse optimization grid. The pseudo-inverse in eq. (6) is deliberately a *regularized* eigen-inverse: eigenvalues below $\text{INFO_RANK_RTOL} \cdot \lambda_{\text{max}}$ are clamped *up* to the cutoff, so a θ -direction the quotes do not identify contributes a large-but-finite handle variance. A strict pseudo-inverse would do the opposite — read an unidentified direction as *zero* uncertainty — which is the one failure mode a filter must never inherit from its measurement.

The covariance is then inflated by realized inconsistency:

$$R_t = \rho_t R_x, \quad \rho_t = \text{clip}\left(\frac{\chi_t^2}{\max(m-d, 1)}, 1, \rho_{\text{max}}\right), \quad \chi_t^2 = r(\hat{\theta})^\top W r(\hat{\theta}), \quad (7)$$

with χ^2 read off the quote rows only and the cap ρ_{max} (`RESID_INFLATION_CAP`) preventing one broken chain from poisoning the state. This is the mechanism that makes close-strike contradictions visible to the filter: a contradictory cluster may have many quotes, but if it cannot be fitted within its stated noise, ρ_t grows and the Kalman gain falls in the directions the cluster does not reliably identify.

Remark 2 (Bands and covariance). For bid–ask mode, quotes inside the band should not pretend to identify the mid. On the Jacobian route this comes for free: inactive hinge rows differentiate to zero inside the spread, so in-band quotes contribute nothing to $J^\top W J$, and only the weak mid anchor remains at its small weight. This matches Note 07’s semantics — inside the spread the market is a set, not a point — with no special-casing, a concrete advantage over the factor route, which must proxy band width through a spread factor.

4.4 Units: information is only information in stated noise

Caution

The production quote weights are *relative* (equal or time-value density, Note 07), not $1/\text{noise}^2$. Read at face value, $J^\top W J$ is therefore the observed information under an implied noise of one full volatility point per quote — so small that R_x saturates the sanity envelope and the filter freezes. The shipped builder fixes the units at the source: the data rows of J and r are divided by the stated per-quote noise standard deviation — the bid–ask half-spread floored at `RMS_FLOOR`, or the haircut — *before* the information and χ^2 are formed, while the intrinsic regularization rows keep their own scale (they are a prior, not a measurement). R_t then obeys the quadratic contract the algebra assumes: doubling the stated noise quadruples the covariance. The same scale s_q reappears twice in active mode (section 6.2): the prediction prior enters the fit at weight $\lambda_j = s_q^2 / \text{Var}(O_j)$, and the posterior information is recovered by unwhitening *all* rows by s_q — one consistent value in both places, or the MAP algebra silently breaks.

4.5 The factor route (fallback and A/B column)

The cheaper builder reuses the existing precision vocabulary,

$$R_t^{-1} = \text{diag}(c_h) \pi_{\text{fit}} f_{\text{density}} f_{\text{spread}} f_{\text{fresh}}, \quad (8)$$

where c_h is per-handle confidence, π_{fit} is inverse squared RMS with a floor and cap, and the scalar factors are exactly those of `volfit/graph/precision.py`. The realized misfit already lives in the

RMS factor, so no separate ρ inflation is applied (it would double-count). This route survives as the fallback when no solution Jacobian was retained (cached fits) and as the sweep’s A/B diagnostic column — which is how the Jacobian route’s advantage got *measured* rather than assumed (section 8).

5 The prediction law and the state’s life cycle

5.1 Transport and process noise

The prediction must transport the previous filtered smile before comparing it with today’s quotes. For a handle state,

$$m_t^- = \mathcal{T}_t(m_{t-1}^+), \quad P_t^- = A_t P_{t-1}^+ A_t^\top + Q_t, \quad (9)$$

where $\mathcal{T}_t$ is the SSR/spot-vol transport of Note 12. Production uses the first-order handle map for a log-forward move h ,

$$\sigma'_{\text{atm}} = \sigma_{\text{atm}} + \text{SSR} \cdot s_{\text{atm}} h, \quad s'_{\text{atm}} = s_{\text{atm}} + \kappa_{\text{atm}} h, \quad \kappa'_{\text{atm}} = \kappa_{\text{atm}}, \quad (10)$$

with $A_t = I$ (the uncertainty growth is carried entirely by Q_t ; a full curve transport is not available for a bare handle vector, and seeding uses the exact curve transport through the prior machinery instead). The process covariance is a diagonal sum,

$$Q_t = Q_{\text{clock}}(\Delta t) + Q_{\text{spot}}(|h|) + Q_{\text{event}} + Q_{\text{source}} + Q_{\text{model}}, \quad (11)$$

with the following meanings:

Clock noise.

Volatility surfaces diffuse through time; the baseline is $q_h^2 \Delta t$ per handle (std = $q_h \sqrt{\Delta t}$). The ATM scale q is the headline knob `filterProcessVolBpSqrtDay`: the design note proposed 10 vol bp/ $\sqrt{\text{day}}$, and the three-regime backtest flipped the shipped default to 30 (section 8).

Spot transport noise.

The larger the log-forward move $h = \log(F_t/F_{t-1})$, the less certain the transported state: std = `filterTransportNoiseScale` · $|h|$ in each handle’s typical move scale. This is the same intuition as the transport-distance factor in prior persistence.

Event noise.

Known event windows widen the prediction before the event resolves. The event clock of the calibration changes the time variable; this term admits that the shape response itself is uncertain.

Source noise.

Switching provider, as-of mode, or live/previous-close source widens or resets the filter. A live quote stream and a prior-close snapshot are not the same stochastic clock.

Model noise.

If the display model changes, the filtered handle state may persist (it is model-agnostic), but model-specific parameters do not.

Caution

Do not filter native local-vol parameters. The local-vol grid (Note 04) is a projection machinery with its own PDE stability and no-arbitrage constraints. Filter the target smile handles, then project or refit the local-vol surface to that target.

5.2 Seeding, resets, and what survives a recalibration

A filter is only as trustworthy as its life-cycle rules, so production pins them down explicitly (`volfit/api/observation_filter.py`):

Seeding.

When a saved prior snapshot exists, the first state is the *transported* prior through the provenance hierarchy of Note 13, with covariance from the provenance-tier precisions. When none exists, the seed is the committed fit’s *own* backbone handles at bootstrap-tier precision — the same information a bootstrap fetch would produce, for free. There is deliberately no hidden extra fit on the seeding path (appendix B records the incident that made this a rule).

Resets.

A manual quote edit moves the session version and resets the node (the edited chain invalidates the measurement the state was built on); a calendar gap beyond `filterResetHours` resets as **stale** (predicting across a long dark period is worse than reseeding from the transported prior); a source or as-of change wipes the store outright (the strict source-reset policy). Every reset records its reason in the diagnostics payload.

Recalibration keeps the state.

The update is sequential — one step per genuinely *new* observation, idempotent per (node, data version, session version). Recalibrating an unchanged snapshot re-reads the stored state; a refetch is a new observation, not a reset.

6 The clean architecture

There are two mathematically safe ways to let the filter act, and production ships both behind `observationFilterMode` (`volfit/api/filter_mode.py`): `off` is byte-identical to the filter never existing, `overlay` computes and displays, `active` steers calibration as a one-stage MAP.

6.1 Overlay mode: compute, display, do not affect calibration

Overlay mode is the pilot delivery. On every committed fit the app layer prepares quotes, extracts (z_t, R_t) from the data-only information, predicts (m_t^-, P_t^-) from the previous state, applies eq. (4) per handle, and stores the full audit trail — prediction, observation, innovation, gain, posterior — for the smile viewer, which draws the raw calibration, the transported prediction and the filtered curve with its credible band. This mode cannot double-count quotes because the posterior never feeds back into the fit; it is therefore the sandbox in which Q_t , R_t and the reset rules were tuned (section 8).

6.2 Active mode: one-stage MAP, not posterior plus quotes

Active mode must not first build m_t^+ from today's quotes and then add m_t^+ as a prior while fitting the same quotes again. That counts q_t twice. The correct active objective is the one-step MAP problem

$$\mathcal{L}_{\text{active}}(\theta) = \mathcal{L}_{\text{quote}}(\theta; q_t) + \frac{1}{2} \|\mathcal{H}(\theta) - m_t^-\|_{(P_t^-)^{-1}}^2 + \mathcal{L}_{\text{intrinsic}}(\theta) + \mathcal{L}_{\text{persist}, \perp}(\theta). \quad (12)$$

Here $\mathcal{H}(\theta)$ extracts the filtered handle coordinates from the model, the second term is the Kalman prediction prior, and the quote likelihood is today's market. In the linear-Gaussian case, minimizing eq. (12) gives the same m_t^+ as eq. (4); in the nonlinear smile case it is the extended-Kalman/MAP version.

Proposition 2 (No double counting in the MAP form). *If the model is linear, the quote extractor is $z = Hx + \epsilon$ with $\epsilon \sim \mathcal{N}(0, R)$, and the prediction is $x \sim \mathcal{N}(m^-, P^-)$, then the minimizer of*

$$\frac{1}{2} \|z - Hx\|_{R^{-1}}^2 + \frac{1}{2} \|x - m^-\|_{(P^-)^{-1}}^2$$

is the Kalman posterior mean in eq. (4).

Proof. The first-order condition is

$$(H^\top R^{-1} H + (P^-)^{-1})x = H^\top R^{-1} z + (P^-)^{-1} m^-.$$

The Woodbury identity rewrites the inverse of the left-hand side as $P^- - P^- H^\top (H P^- H^\top + R)^{-1} H P^-$. Substitution gives $x = m^- + P^- H^\top (H P^- H^\top + R)^{-1} (z - H m^-)$, which is eq. (4). $\square$

Production realizes eq. (12) with four locked decisions (`build_filter_prior` in `volfit/calib/observation_filter.py` and the MAP bookkeeping in `volfit/api/observation_filter.py`):

1. **The prediction prior is an *ungated* operator target.** It reuses the smile-factor stencil legs at $k \in \{-h, 0, h\}$ with $h = \text{FILTER_STENCIL_H} = 0.06$: on a locally quadratic smile the stencil identities $\sigma(h) - \sigma(-h) = 2h s_{\text{atm}}$ and $\sigma(h) - 2\sigma(0) + \sigma(-h) = h^2 \kappa_{\text{atm}}$ are *exact*, so the handle targets convert to stencil targets with no approximation beyond local quadraticity. Because these are ordinary operator residuals, the prior flows to LQD, SVI and Multi-Core SIV through the existing plumbing with zero per-model wiring — and, crucially, it carries *no* activation gate. The Kalman prior is always on at its covariance weight; that is precisely the persistence/filter distinction of section 2.
2. **The weights are the MAP objective in the fit's own units.** The quote rows are (near-)unit-weighted vol errors, i.e. the true MAP objective $\sum_i (e_i/s_q)^2 + \sum_j d_j^2/P_{jj}^-$ multiplied through by s_q^2 , so the consistent prior weight is $\lambda_j = s_q^2/\text{Var}(O_j)$ with s_q the node's typical stated per-quote noise (the median bid-ask half-spread, floored — the same s_q as section 4.4).
3. **Persistence auto-exclusion is hard-coded.** When the mode is active, `resolve_prior_mode` (`volfit/api/prior_mode.py`) drops every persistence builder overlapping the handle coordinates — operators, smile factors, the near-ATM strike anchor. What survives is exactly what the filter state does not carry: the deep-tail strike anchor and the graph's dark-node baseline. There is deliberately no knob; two anchors to the same previous state would violate invariant 3.

4. **No second update.** The committed fit *is* the one-stage MAP solution, so m^+ is simply its backbone handles, and P^+ comes from unwhitening the full solver information (data rows plus whitened prior rows) by the same s_q — landing exactly at $\mathcal{I}_{\text{data}} + (P^-)^{-1}$, the posterior information of proposition 2. Applying eq. (4) again against the same quotes would double-count them; the test suite locks the MAP-equals-Kalman identity to 10^{-10} . One guard is added on top: P^+ is capped at P^- per handle, because information only adds — inconsistent data ($\rho > 1$) should drive P^+ *toward* the prediction uncertainty, never beyond it.

6.3 Where prior persistence remains

Prior persistence remains available in active mode, with explicit semantics: it acts on deep tails and operator baskets not represented in the Kalman state; it provides the saved prior from which the first filtered state is seeded; and it acts on dark graph nodes that have no current quote observation. It does not also anchor ATM/skew/curvature to the same previous state that already appears as m_t^- in the Kalman term — the auto-exclusion above makes this structural rather than procedural.

7 Two case files

The first case file is the note’s original design vignette, now computed by the production update (`gen_kalman.py` feeds the numbers below through `kalman_update`; fig. 1). The second is a real incident from the temporal backtest — the one that changed the production update.

Case file: close-strike contradiction versus true gap

Setup. A one-month equity smile has nine near-ATM quotes and no reliable quotes beyond 25Δ . The previous filtered state transported to today is

$$m^- = (20.0\%, -0.35, 0.10), \quad \sqrt{\text{diag } P^-} = (0.30\%, 0.08, 0.05).$$

The near-ATM quotes imply a data-only handle observation

$$z = (20.4\%, -0.37, 0.55).$$

The ATM level and skew are consistent, but one stale close strike forces a very large curvature if the fit chases mids literally.

Diagnosis. Quote density alone would say the region is well observed. Therefore prior persistence must switch off for ATM/skew/curvature; using it to pull curvature back would violate the Note 13 rule. The contradiction instead appears as measurement noise, through the χ^2 inflation of eq. (7): the observation standard deviations come out as

$$\sqrt{\text{diag } R} = (0.15\%, 0.05, 0.30).$$

The production per-handle gains are

$$K_\sigma = 0.80, \quad K_s = 0.72, \quad K_\kappa = 0.027.$$

Filter. The update moves the real observed level and skew, $m_\sigma^+ = 20.32\%$ and $m_s^+ = -0.364$, but barely accepts the isolated curvature kink: $m_\kappa^+ = 0.112$.

Persistence. The unquoted wing is a different matter. For a deep-tail functional, observation precision is below requirement, so the activation gate of eq. (1) is near one and the transported prior supplies the tail anchor. The same day therefore has both effects: Kalman filtering denoises the observed cluster, while prior persistence fills the wing gap.

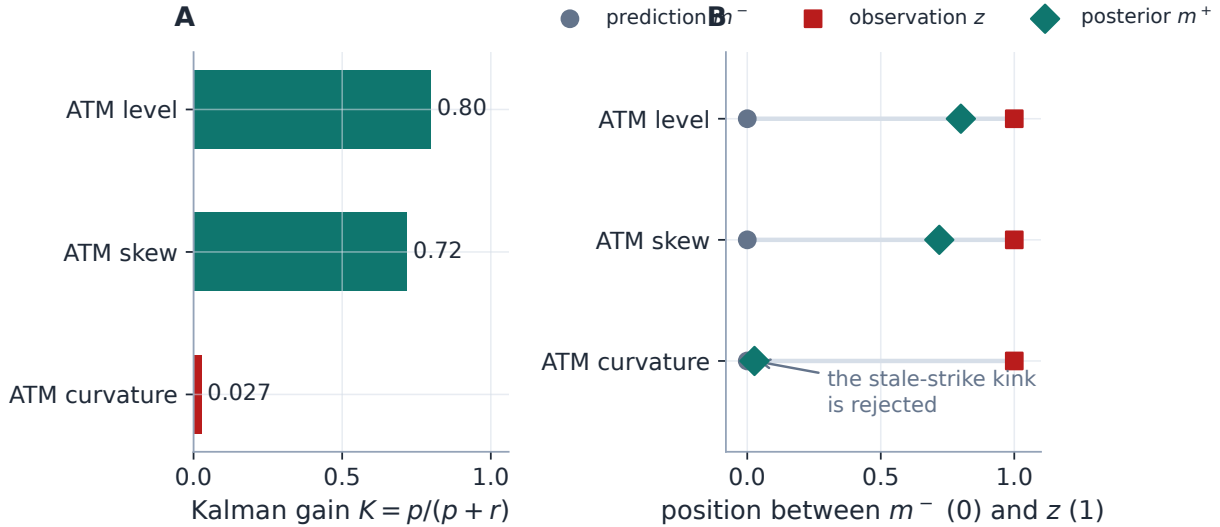


Figure 1: The case file through the production update. (A) The tight level and skew observations are accepted at gains 0.80 and 0.72; the contradiction-inflated curvature passes at 0.027. (B) Each posterior lands on the prediction→observation segment exactly at its gain: the level moves to the market, the stale-strike curvature kink is rejected — without any gap logic ever firing.

This example is deliberately small, but it captures the implementation rule: a dense cluster with a bad internal residual is not a gap; it is a high- R_t observation. A strike range with no reliable quotes is a gap; it belongs to prior persistence.

Case file: the off-diagonal blow-up on coarse chains

Setup. The Phase-5 temporal backtest replayed the August 2024 spike day pair across the eight pilot assets in overlay mode, full-covariance update, Jacobian route — the design configuration of eq. (4) verbatim.

Failure mode. On EEM and EFA — coarse-strike, wide-spread ETF chains — the posterior ATM error reached 3–28 *vol points*, worse than *both* baselines (the raw measurement and the gain-zero prediction). A scalar update cannot do that: by proposition 1 its posterior is a convex combination of prediction and observation. The blow-up had to be a cross-handle effect.

Diagnosis. On a coarse chain the Jacobian covariance eq. (6) carries strong level–curvature correlations (few strikes identify level and curvature jointly, not separately). A junk curvature innovation — the very thing the filter is supposed to swallow — then drags the ATM level through the *off-diagonal* entries of $K = P^-S^{-1}$. The pilot cap is no defence: `filterMaxGain` caps the own-gains $\text{diag}(KH)$ only, and the damage travels through the off-diagonal terms it

never touches.

Fix. Production diagonalizes P^- and R before the update (`DIAGONAL_UPDATE` in `volfit/api/observation_filter.py`): per-handle scalar gains, the Note 14 graph convention (remark 1). The full-covariance update remains available for later study — the correlations are real information, and using them safely (e.g. through a correlation shrinkage) is future work, not a revert.

Verdict. Post-fix, EEM wins against the raw measurement (4.5 vs 8.1 bp, ζ mean 0.14) and EFA degrades gracefully — near-zero gain from its wide-spread R , ζ mean 0.26, i.e. conservative rather than wrong. The behaviour is locked by `test_observation_filter_app.py`.

8 What the backtest measured

The filter was not accepted because charts look smooth. The temporal harness `backend/backtest/observation_filter.py` drives the *production* commit path over consecutive captured day pairs in three market regimes (the August 2024 spike, October 2022 high-vol, July 2023 low-vol), eight assets each: fit day $T-1$'s full chain and commit it through the filter; carry the state into day T with the real snapshot-timestamp Δt and forward transport; build day T 's measurement from a *thinned* ATM-only chain under a scenario; score the posterior against the day- T full-chain truth and against *two* baselines — the raw measurement (no filter) and the pure transported prediction (gain 0). Three scenarios probe the three failure modes: **thinned** (plain consecutive snapshots), **contradiction** (two adjacent near-ATM strikes kinked in opposite directions — curvature noise that must be rejected), and **shock** (a true +5 vol-point jump with unchanged spreads — a move that must be followed). Calibration of uncertainty is scored by the standardized residual $\zeta = (\text{truth} - m^+)/\sqrt{P^+ + R_{\text{truth}}}$ — the truth is itself a fitted estimate, and scoring against $\sqrt{P^+}$ alone overstated miscalibration threefold. Results are bucketed by maturity (≤ 30 and > 30 days to expiry); the full sweep spans 38 181 scored steps. Figure 2 shows the two headline panels; the numbers below are all at the decision cell (Jacobian route, thinned, $> 30\text{d}$) unless stated.

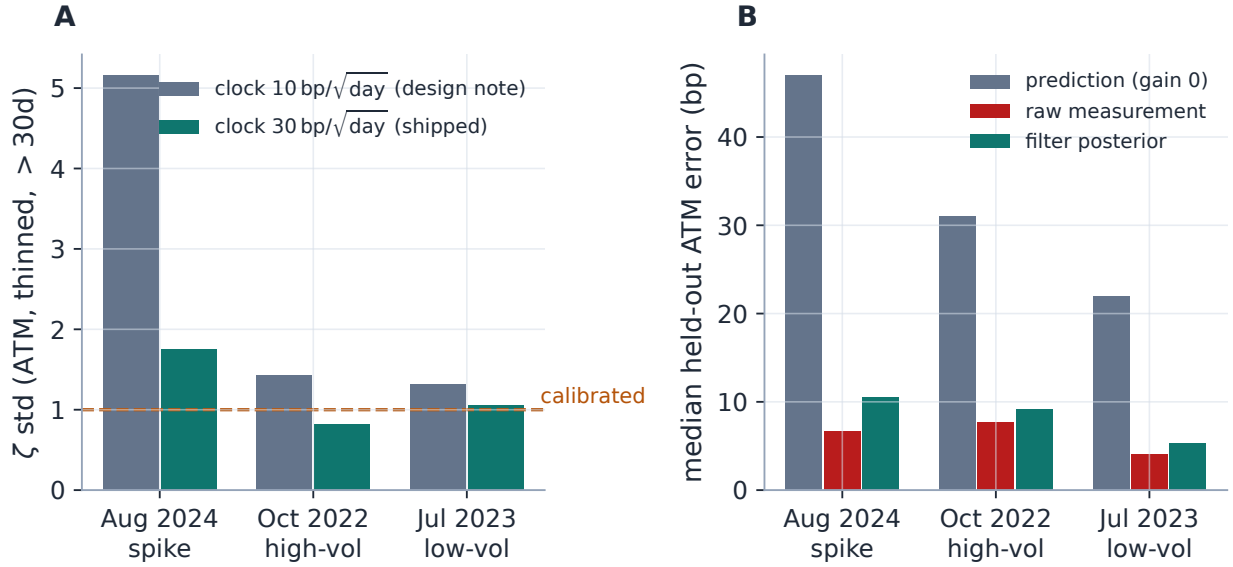


Figure 2: (A) Tripling the clock noise collapses the ζ standard deviation toward the calibration target of 1 in every regime (5.16 $\rightarrow$ 1.75, 1.43 $\rightarrow$ 0.82, 1.32 $\rightarrow$ 1.05): the design value of 10 bp/ $\sqrt{\text{day}}$ starved the prediction uncertainty. (B) On plain thinned days the posterior sits between the raw measurement and the gain-zero prediction: the filter pays for itself on the noisy tail, not on clean liquid days — exactly the success criterion.

The clock-noise default flipped (10 $\rightarrow$ 30). The evidence is one-sided: in every regime, on both covariance routes, at every scenario, $q = 30$ bp/ $\sqrt{\text{day}}$ improves on the design value of 10 — the ζ std collapses toward 1 (panel A), shock lag shrinks 3–7 $\times$, and win rates rise. A too-small clock noise starves P^- , so the filter over-trusts its own prediction and both lags real moves and reports overconfident posteriors. The shipped default is 30.

Jacobian versus factors is a real trade-off; Jacobian stays the default. On contradiction — the filter’s core denoising case — the Jacobian route is 2–3 $\times$ better in every regime (high-vol: 10.5 vs 29.7 bp; spike: 15.8 vs 45.9; low-vol: 6.3 vs 21.7): its geometry-aware R sees which directions the kinked cluster fails to identify. On shock the ranking reverses (spike: 38.6 vs 18.7 bp lag) because the factor route’s blunter, smaller effective R yields higher gain. Contradiction rejection is the feature’s purpose, the shock gap closes for both routes under the adaptive- Q item below, and the factor route stays one knob away.

The honest median-day statement. On plain consecutive days the raw measurement is already good (4.0–7.7 bp median ATM error across regimes) and the filter’s median win rate is 0.38–0.49 — below one half. The posterior (10.5/9.2/5.3 bp per regime) beats the gain-zero prediction (47/31/22 bp) everywhere but does not beat a clean measurement on a clean day, and it should not: the filter pays for itself on the noisy tail — the contradiction columns, the illiquid names, the wide-spread chains — which is precisely the success criterion set before the backtest ran (“lower held-out error in noisy snapshots”, not “lower every RMS”). The slightly negative ζ means on thinned days are a harness artifact worth recording: the ATM-window thinning is mildly biased against the full-chain truth.

Documented limitations and the pre-active work list. Three items are measured, known, and deliberately not hidden. (i) The ≤ 30 DTE bucket is a different regime: short-dated nodes carry 90–160 bp thinned-vs-full discrepancies from quote and de-Americanization noise (the Note 04/05 short-dated diagnosis, not a filter defect), the filter loses to raw there, and the buckets are reported separately so neither regime masks the other; the candidate policies are excluding < 30 DTE nodes or a maturity-scaled measurement-noise floor. (ii) A fixed clock cannot span calm and spike regimes — the pre-active follow-up is an adaptive Q (innovation-gated widening), which is what closes the residual shock lag on both routes. (iii) The **active** mode should join the harness sweep before any active-mode default is considered.

9 What is original here

The Kalman equations are classical [1, 2]. The Vol-Fitter-specific contribution is the separation of three notions that are usually blended in volatility tools:

1. **Quote likelihood:** today’s prepared quotes, with bands, weights and no-arbitrage model structure (Note 07).
2. **Temporal filtering:** a transported latent smile state with prediction and measurement covariance.
3. **Gap persistence:** a prior that activates only where current quotes do not identify a functional (Note 13).

That separation is what makes the feature safe: the filter denoises a noisy observed smile without using yesterday as a fake quote, and prior persistence stabilizes unobserved wings without damping a real current-market move. Two implementation pieces are worth singling out beyond the architecture. The *measurement covariance in stated noise* (section 4.4): tying $J^T W J$ to the bid–ask half-spread is what makes R_t a market object rather than a fit artifact, and the same scalar s_q then makes the active MAP weights and the posterior information mutually consistent. And the *ungated stencil prior* (section 6.2): expressing (m^-, P^-) as exact ATM-stencil operator targets lets one prediction prior steer every parametric model through existing plumbing, while the hard-coded persistence auto-exclusion makes double-anchoring impossible rather than discouraged.

10 Limitations

The diagonal update discards genuine cross-handle information by design; the off-diagonal blow-up of section 7 showed why that trade is currently right, but a shrunk-correlation update is a live research item, not a closed door. The clock process noise is a fixed scalar and demonstrably cannot span calm and spike regimes at once ($q = 30$ is the best single value, not a resolution — adaptive Q is the follow-up). Short-dated nodes (≤ 30 DTE) are dominated by quote and de-Americanization noise that the filter can neither model nor fix; they need their own measurement-noise policy before active mode is trusted there. The state is per-node: a graph-coupled filter (Note 14’s block covariance with time dynamics) is deferred until the single-node filter has earned trust. And the measurement carrier is the LQD backbone: a displayed model whose handles disagree with the backbone’s inherits that disagreement through the overlay.

11 Traceability

Table 1: Claims in this note and the code/tests that lock them.

Claim	Object	Code anchor	Test anchor
Joseph update PSD for any gain; own-gain cap; scalar shrinkage; agrees with the graph posterior to 10^{-12}	eq. (4), proposition 1	volfit/calib/observation_filter.py	test_observation_filter.py
One-stage MAP minimizer = Kalman posterior to 10^{-10} ; ungated stencil prior; no second update	proposition 2, eq. (12)	volfit/calib/observation_filter.py, volfit/api/observation_filter.py	test_observation_filter.py, test_filter_active.py
Jacobian covariance; regularized eigen-inverse; stated-noise units contract	eq. (6)	volfit/calib/observation_measurement.py	test_observation_measurement.py
Contradictions inflate R ; band rows contribute nothing inside the spread	eq. (7)	volfit/calib/observation_measurement.py	test_observation_measurement.py
Diagonal update; idempotent commits; reset matrix; seeding without a hidden fit	remark 1, section 5.2	volfit/api/observation_filter.py	test_observation_filter_app.py
Mode resolution; off byte-identical; overlay never steers fits	section 6	volfit/api/filter_mode.py	test_filter_mode.py
Persistence auto-exclusion: only the deep-tail anchor and dark nodes survive in active mode	section 6.2	volfit/api/prior_mode.py	test_filter_active.py
Temporal backtest protocol, scenarios and ζ scoring	section 8	backend/backtest/observation_filter.py	test_filter_backtest.py

A Hyperparameter atlas

Every knob touching the observation filter, with its shipped default and role. Surfaced knobs are user-facing `OptionsSettings`; hidden knobs are internal constants tuned to the algorithm.

Table 2: Observation-filter hyperparameters, as shipped.

Knob	Default	Role
<i>Surfaced (OptionsSettings)</i>		
<code>observationFilterMode</code>	<code>off</code>	<code>off</code> / <code>overlay</code> / <code>active</code> (section 6); <code>off</code> is byte-identical to the feature never existing.
<code>filterCovarianceMode</code>	<code>jacobian</code>	Measurement-covariance route: <code>jacobian</code> eq. (6) (backtested default, section 8) or <code>factors</code> eq. (8) (fallback + A/B column).
<code>filterProcessVolBpSqrtDay</code>	30	ATM clock process noise in vol bp per $\sqrt{\text{day}}$. Backtested: the design value 10 starved P^- in every regime; 30 is one-sided better (fig. 2A).
<code>filterProcessSkewSqrtDay</code>	0.02	Skew clock process-noise scale.
<code>filterProcessCurvSqrtDay</code>	0.05	Curvature clock process-noise scale.

Knob	Default	Role
<code>filterTransportNoiseScale</code>	0.10	Extra process std per unit $ \log(F_t/F_{t-1}) $ transport distance (eq. (11) spot term).
<code>filterResidualInflation</code>	on	Apply eq. (7) so a dense-but-contradictory cluster reads as measurement noise.
<code>filterMaxGain</code>	1.0	Pilot safety cap on the per-handle own-gains; at 1.0 it never binds (own-gains sit in $(0, 1)$ by construction).
<code>filterResetHours</code>	96	Maximum data gap the filter predicts across; longer gaps reset as stale (the default spans a weekend plus a holiday).
<code>filterDataOnlyPrepass</code>	off	Opt-in extra data-only fit so z_t is strictly persistence-free; off reuses the committed fit and flags contamination instead.
<i>Hidden (internal constants)</i>		
<code>DIAGONAL_UPDATE</code>	on	Per-handle scalar gains (remark 1); the measured answer to the off-diagonal blow-up case file.
<code>FILTER_STENCIL_H</code>	0.06	ATM stencil half-width of the active MAP prior legs; the stencil identities are exact on a locally quadratic smile.
<code>RESID_INFLATION_CAP</code>	25	Cap on ρ_t in eq. (7): one broken chain cannot poison the state.
<code>INFO_RANK_RTOL</code>	10^{-10}	Relative eigenvalue cutoff of $\mathcal{I}_\theta$; small eigenvalues are clamped <i>up</i> , so unidentified directions inflate R finitely instead of exploding or vanishing.
<code>CHOL_JITTER</code>	10^{-12}	Base jitter of the whitening Cholesky; any nonzero use is reported as a diagnostic event, never silent.
<code>HANDLE_MOVE_SCALES</code>	(0.03, 0.05, 0.5)	Typical one-day move scales giving the dimensionless transport knob a per-handle magnitude (the Note 14 units convention).
<code>RMS_FLOOR</code>	10^{-4}	Floor on the stated per-quote noise (one vol bp): an excellent fit is not infinitely precise. Chains without band data use 10× this floor.
variance envelope	graph floors/-caps	Per-handle R diagonals are clipped into $[1/\text{cap}, 1/\text{floor}]$ from <code>volfit/graph/precision.py</code> , correlations rescaled to survive.
<code>_filter_version</code>	—	Overlay-only knob changes bump a lightweight version so the overlay refreshes without invalidating any fit cache; only active -affecting changes bump the options version.

B Performance notes

1. **Overlay mode is invisible next to calibration.** The update is a dense $d \times d$ solve with $d = 3$; state storage is $O(d^2)$ per node.
2. **The Jacobian covariance is nearly free.** $J^\top W J$ comes from the solution Jacobian the calibrators already retain, and the handle Jacobian G is $2P$ slice *builds* (no fitting) on the coarse

optimization grid — about $4\times$ cheaper per build than the display quadrature.

3. **Seeding must never hide a fit.** The first implementation seeded through the prior bootstrap branch, whose fetch path silently ran a *full extra mid calibration per node*; switching the filter on made a live universe crawl. The shipped rule: seed from the transported saved prior when one exists, else from the committed fit’s own handles at bootstrap-tier precision — the same information, for free. Reverted-and-redesigned, per house practice.
4. **Overlay curves are memoized per committed state.** The filtered curve is the backbone retargeted to m^+ (a Newton solve, Note 01); computing it per GET made the live UI feel frozen, since the smile view polls the payload on every refresh signal. It is now computed once per committed state.
5. **Active MAP adds no second fit pass.** The prediction prior is three residual rows in the existing least-squares stack; the optional data-only prepass is the only real cost lever and is off by default.
6. **Graph generalization is natural but later.** A graph Kalman filter replaces the per-node P_t^- with a block covariance over the universe — Note 14’s covariance-form posterior with time dynamics. It comes after the single-node filter has earned trust.

C Reference implementation

The numerical heart is small and model-free: it assumes the caller has already built the prediction and measurement moments. The listing below is executed by `gen_kalman.py` against the production `volfit.calib.observation_filter.kalman_update` on a random-seeded 3×3 problem (full covariances, $H = I$); the maximum deviation across mean, covariance, innovation, innovation covariance and gain is 0 — machine precision, as it should be, since production adds only input validation, the own-gain cap and the PSD guard around the same algebra.

Listing 2: Covariance-form Kalman update with Joseph covariance (verified against production; agreement stated above).

```

1  def kalman_update_ref(mean_pred, cov_pred, obs, obs_cov, H=None):
2      m = np.asarray(mean_pred, dtype=float)
3      P = np.asarray(cov_pred, dtype=float)
4      z = np.asarray(obs, dtype=float)
5      R = np.asarray(obs_cov, dtype=float)
6      H = np.eye(m.size) if H is None else np.asarray(H, dtype=float)
7      innovation = z - H @ m
8      S = H @ P @ H.T + R
9      K = np.linalg.solve(S.T, (P @ H.T).T).T
10     out_mean = m + K @ innovation
11     I_KH = np.eye(m.size) - K @ H
12     out_cov = I_KH @ P @ I_KH.T + K @ R @ K.T
13     return out_mean, 0.5 * (out_cov + out_cov.T), innovation, S, K

```

The active-mode residual rows corresponding to eq. (12) are equally small once the model exposes a handle extractor:

Listing 3: Whiten residual rows for the active MAP prediction prior (distilled from `volfit/calib/observation_filter.py`).

```
1 def prediction_prior_residual(model_handles, mean_pred, cov_pred):  
2     L = np.linalg.cholesky(np.asarray(cov_pred, dtype=float))  
3     return np.linalg.solve(L, np.asarray(model_handles) - np.asarray(  
        mean_pred))
```

In production the Cholesky solve uses a small escalating jitter and reports it; adding jitter is a diagnostic event, not a silent repair.

References

- [1] R. E. Kalman. A new approach to linear filtering and prediction problems. *Journal of Basic Engineering*, 82(1):35–45, 1960.
- [2] B. D. O. Anderson and J. B. Moore. *Optimal Filtering*. Prentice-Hall, 1979.
- [3] A. Gelman et al. *Bayesian Data Analysis*. CRC Press, 3rd ed., 2013.
- [4] J. Gatheral. *The Volatility Surface*. Wiley, 2006.